CO Assignment 4

Pipelining

Name: Arnav Singhal
MIS: 612203012
SY B. Tech. Comp. Div 1

Problem Statement:

Identify and understand the dependencies by taking set of instructions without data forwarding

Set 1 of instructions:

L.D F6,32(R2)

L.D F2,44(R3)

MUL.D F16,F2,F4

SUB.D F8,F2,F6

DIV.D F10,F16,F6

ADD.D F6,F8,F2

Set 2 of instructions:

I1 : ADD R1, R2, R3

I2 : MUL R7, R1, R3

I3 : SUB R4, R1, R5

I4 : ADD R3, R2, R4

I5 : MUL R7, R8, R9

Identify and understand the dependencies by taking set of instructions with data forwarding

Set 1 of instructions:

L.D F6,32(R2)

L.D F2,44(R3)

MUL.D F16,F2,F4

SUB.D F8,F2,F6

DIV.D F10,F16,F6

ADD.D F6,F8,F2

Set 2 of instructions:

I1 : ADD R1, R2, R3

I2 : MUL R7, R1, R3

I3 : SUB R4, R1, R5

I4 : ADD R3, R2, R4

I5 : MUL R7, R8, R9

INTRODUCTION

What is Pipelining?

Pipelining is a technique used in modern processors to improve instruction throughput. It breaks down the execution of instructions into distinct stages – Fetch, Decode, Execute, Memory Access and Write Back. Each stage is handled by a different part of the processor, allowing multiple instructions to be in different stages of execution simultaneously, just like an assembly line.

<u>How it Makes Execution More Efficient:</u>
Without pipelining, the CPU would complete one instruction at a time before starting the next. With pipelining, it starts a new instruction in every clock cycle while the previous ones are still in progress. This increases instruction throughput (more instructions completed per unit time) and makes much better use of CPU resources. Ideally, a 5-stage pipeline can complete one instruction per clock cycle after the initial fill-up phase.

Data Forwarding

Data forwarding, also known as bypassing, is a technique used to resolve data hazards in pipelining. A data hazard occurs when an instruction depends on the result of a previous instruction that hasn't completed yet.

Instead of waiting for the result to be written back to the register file, data forwarding routes the result directly from one pipeline stage to another—usually from the Execute or Memory stage to the Decode or Execute stage of the next instruction.

This eliminates unnecessary stalls and keeps the pipeline flowing smoothly.

Pipelining with Data Forwarding vs Pipelining without Data Forwarding

Aspect	Without Data Forwarding	With Data Forwarding
Hazard Handling	Requires stalling to wait for data availability	Resolves many hazards by directly routing data
Performance	Slower due to frequent stalls	Faster due to reduced need for stalls

Control Complexity	Simpler hardware	More complex forwarding logic needed
Instruction Throughput	Lower	Higher

Pipelining without data forwarding

Simulator URL: https://www.ecs.umass.edu/ece/koren/architecture/windlx/main.html

Running the simulator with the first set of instructions:

L.D F6,32(R2)

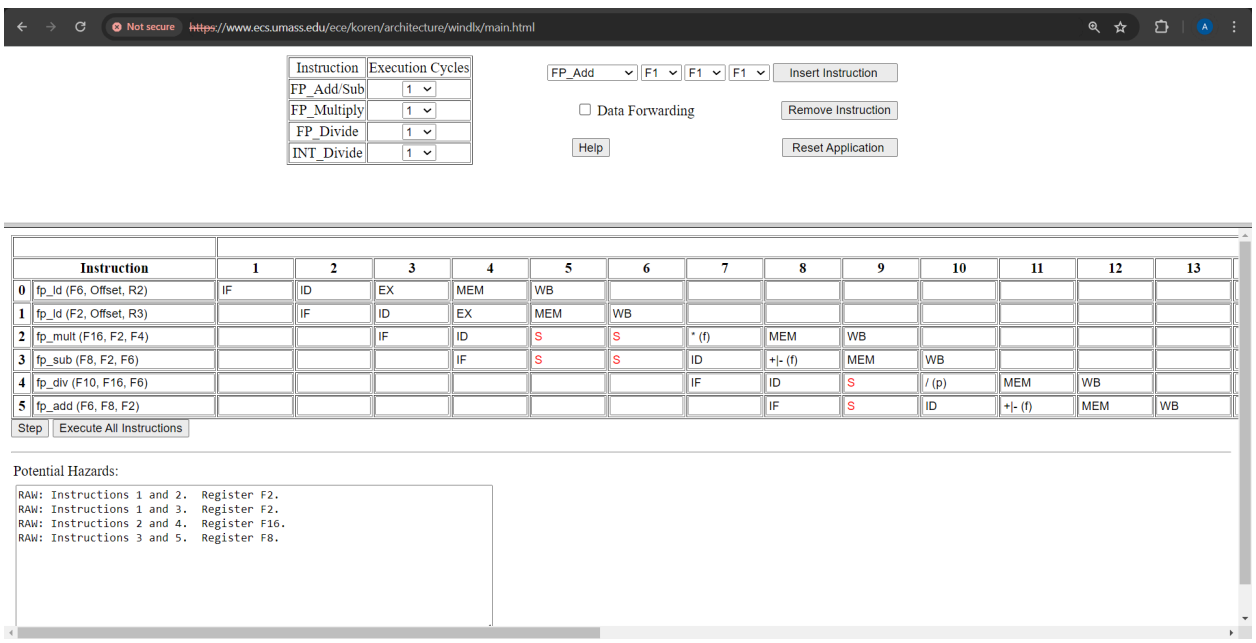
L.D F2,44(R3)

MUL.D F16,F2,F4

SUB.D F8,F2,F6

DIV.D F10,F16,F6

ADD.D F6,F8,F2



Explanation:

1) Here, instruction 2 has F2 as one of its operands. However, the value of F2 will be changed after execution of instruction 1. Hence execution of instruction 2 must be stalled for 2 clock cycles (5 and 6) until the value of F2 is ready (after writeback stage of instruction 1).
2) When the instruction fetch stage is completed for instruction 3, the previous instruction is still in the decode phase due to the execution phase being stalled

for two cycles. Hence, for instruction 3, the decode phase must be stalled for 2 cycles (5 and 6).

3) Instruction 4 has F16 as one of its operands. However, F16 will store the result of the multiplication operation performed in instruction 2. Execution of instruction 4 will have to be stalled for one clock cycle (9) until the writeback stage of instruction 2 is completed.
4) Since execution of instruction 4 is stalled in clock cycle 9, the decode stage of instruction 5 will be stalled in the same clock cycle.

Running the simulator with the second set of instructions:

I1 : ADD R1, R2, R3

I2 : MUL R7, R1, R3

I3 : SUB R4, R1, R5

I4 : ADD R3, R2, R4

I5 : MUL R7, R8, R9

Instruction	Execution Cycles
FP_Add/Sub	1
FP_Multiply	1
FP_Divide	1
INT_Divide	1

INT_Multiply R1 R1 R1 Insert Instruction

Data Forwarding Remove Instruction

Help Reset Application

		CPU Cycles												
	Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13
0	int_add (R1, R2, R3)	IF	ID	+|- (i)	MEM	WB								
1	int_mult (R7, R1, R3)		IF	ID	S	S	* (i)	MEM	WB					
2	int_sub (R4, R1, R5)			IF	S	S	ID	+|- (i)	MEM	WB				
3	int_add (R3, R2, R4)						IF	ID	S	S	+|- (i)	MEM	WB	
4	int_mult (R7, R8, R9)							IF	S	S	ID	* (i)	MEM	WB

Step Execute All Instructions

Potential Hazards:

```
RAW: Instructions 0 and 1.  Register R1.
RAW: Instructions 0 and 2.  Register R1.
RAW: Instructions 2 and 3.  Register R4.
```

Pipelining with data forwarding

Simulator URL: https://www.ecs.umass.edu/ece/koren/architecture/windlx/main.html

Running the simulator with the first set of instructions:

L.D F6,32(R2)

L.D F2,44(R3)

MUL.D F16,F2,F4

SUB.D F8,F2,F6

DIV.D F10,F16,F6

ADD.D F6,F8,F2

Instruction	Execution Cycles
FP_Add/Sub	1
FP_Multiply	1
FP_Divide	1
INT_Divide	1

FP_Add | F1 | F1 | F1 | Insert Instruction

☑ Data Forwarding | Remove Instruction

Help | Reset Application

	Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13
0	fp_ld (F6, Offset, R2)	IF	ID	EX	MEM	WB								
1	fp_ld (F2, Offset, R3)		IF	ID	EX	MEM	WB							
2	fp_mult (F16, F2, F4)			IF	ID	S	* (f)	MEM	WB					
3	fp_sub (F8, F2, F6)				IF	S	ID	+|- (f)	MEM	WB				
4	fp_div (F10, F16, F6)						IF	ID	/ (p)	MEM	WB			
5	fp_add (F6, F8, F2)							IF	ID	+|- (f)	MEM	WB		

Step | Execute All Instructions

Potential Hazards:

```
RAW: Instructions 1 and 2.  Register F2.
```

Running the simulator with the second set of instructions:

I1 : ADD R1, R2, R3

I2 : MUL R7, R1, R3

I3 : SUB R4, R1, R5

I4 : ADD R3, R2, R4

I5 : MUL R7, R8, R9

Instruction	Execution Cycles
FP_Add/Sub	1
FP_Multiply	1
FP_Divide	1
INT_Divide	1

INT_Multiply | R1 | R1 | R1 | Insert Instruction

☑ Data Forwarding | Remove Instruction

Help | Reset Application

		CPU Cycles												
	Instruction	1	2	3	4	5	6	7	8	9	10	11	12	13
0	int_add (R1, R2, R3)	IF	ID	+|- (i)	MEM	WB								
1	int_mult (R7, R1, R3)		IF	ID	* (i)	MEM	WB							
2	int_sub (R4, R1, R5)			IF	ID	+|- (i)	MEM	WB						
3	int_add (R3, R2, R4)				IF	ID	+|- (i)	MEM	WB					
4	int_mult (R7, R8, R9)					IF	ID	* (i)	MEM	WB				

Step | Execute All Instructions

Potential Hazards:

```
No Hazards Found.
```

CONCLUSION:

1. In this assignment, we explored how pipelining improves instruction throughput by overlapping different stages of execution.
2. By analyzing simulations both with and without data forwarding, we observed how data hazards impact performance and that data forwarding effectively reduces unnecessary stalls by routing results directly between pipeline stages.
3. These examples highlighted the practical benefits of data forwarding in maintaining pipeline efficiency, ultimately enabling faster and more optimized execution in modern processors.